

RAG Parsing and Chunking Notes

Structured Study Guide with Code Examples and Interview Questions

Table of Contents

1. RAG Ingestion Pipeline: From Raw Documents to Intelligent Search
2. 1. Unstructured Parsing
3. 2. Structured Parsing
4. 3. Metadata Extraction
5. 4. Content Cleaning
6. 5. Content Normalization
7. 6. CharacterTextSplitter
8. 7. RecursiveCharacterTextSplitter
9. 8. TokenTextSplitter
10. 9. MarkdownHeaderTextSplitter
11. 10. HTMLHeaderTextSplitter
12. 11. Code Splitter
13. 12. Semantic Chunking
14. 13. Chunk Size: The Most Important Parameter
15. 14. Chunk Overlap: Preserving Context at Boundaries
16. 15. Putting It All Together: Production Strategy

RAG Ingestion Pipeline: From Raw Documents to Intelligent Search

When we build an AI-powered search system using Retrieval-Augmented Generation (RAG), the most critical but often overlooked part is **how we prepare the data**. Before any embedding model or LLM ever sees a document, it goes through a carefully designed ingestion pipeline. This guide walks through each stage of that pipeline, explaining what it does, why it matters, and how to implement it well.

The full pipeline looks like this:

```
TEXT
Raw Documents (PDF, DOCX, HTML, CSV, JSON, Email)
  |
  v
Parsing (extract text from raw formats)
  |
  v
Metadata Extraction (capture source, page, department, etc.)
  |
  v
Content Cleaning (remove noise like headers, footers, page numbers)
  |
  v
Content Normalization (standardize dates, names, abbreviations)
  |
  v
Chunking (split into right-sized pieces)
  |
  v
Embeddings (convert chunks into vectors)
  |
  v
Vector Database (store for fast similarity search)
  |
  v
Retriever + LLM (answer user queries)
```

Every step in this chain affects the final answer quality. If parsing is poor, the LLM gets garbage. If chunking is bad, the retriever finds irrelevant results. Let's understand each stage properly.

1. Unstructured Parsing

What It Means

Most real-world business knowledge lives inside PDFs, Word documents, emails, HTML pages, contracts, and research papers. These files don't have neat rows and columns - they have paragraphs mixed with headers, footers, images, tables, page numbers, and formatting. Unstructured parsing is the process of pulling out the actual useful text from all this noise.

Think of it this way: a PDF looks beautiful to a human reader, but to a computer, it's just a blob of layout instructions. The parser's job is to figure out which parts are meaningful content and which parts are just decoration.

Why It Matters

Without good parsing, everything goes wrong downstream:

- **Noise enters your embeddings.** If "Company Confidential" and "Page 3" get embedded alongside actual policy text, your vectors become meaningless.
- **Retrieval accuracy drops.** The search might match on "Page 1" instead of the actual answer.
- **The LLM hallucinates more.** Noisy context confuses the model into generating wrong answers.

With proper parsing, you get clean, meaningful text that leads to accurate embeddings, precise retrieval, and reliable LLM responses.

Challenges in Unstructured Parsing

Real documents are messy. Common challenges include:

- **Mixed content:** A single page may have headings, paragraphs, tables, images, and footers all together.
- **Layout complexity:** Multi-column PDFs can scramble text during extraction.
- **Tables:** Many parsers struggle to preserve table row/column structure.
- **Images:** Text parsers cannot read diagrams or charts - OCR may be needed.
- **Repeated noise:** Headers and footers appear on every single page and must be identified and removed.

A Practical Example

Imagine you load a security policy PDF. The raw extraction gives you:

```
TEXT
Company Confidential
Page 1
Security Policy
Passwords must be changed every 90 days.
Company Confidential
Page 2
Multi-factor authentication is mandatory.
```

After proper parsing, you get just the useful content:

```
TEXT
Security Policy
Passwords must be changed every 90 days.
Multi-factor authentication is mandatory.
```

The repeated headers, footers, and page numbers are gone.

Element-Based Parsing

Modern parsers don't just dump all text - they identify document elements:

```
TEXT
Title Element: "Security Policy"
Paragraph Element: "Passwords must be changed every 90 days."
Table Element: (structured table data)
List Element: (bullet points)
```

This is much better than treating the whole document as one giant text block because it allows smarter chunking later.

Tools You'll Use

LangChain provides several loaders for unstructured data:

```
PYTHON
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_community.document_loaders import Docx2txtLoader
from langchain_community.document_loaders import UnstructuredHTMLLoader
from langchain_community.document_loaders import WebBaseLoader
```

For enterprise-grade parsing, the `unstructured` library is popular because it can identify document elements (title, paragraph, table, list, header, footer) rather than treating everything as flat text.

Common Sources of Unstructured Data

- **PDFs:** Policies, contracts, research papers
- **DOCX files:** SOPs, design docs, manuals
- **HTML files:** Documentation sites, knowledge bases
- **Emails:** Support requests, incident reports
- **Chat data:** Slack, Teams, customer support logs
- **Scanned documents:** Invoices, signed contracts (need OCR)

Interview Questions

Q1. What is Unstructured Parsing?

Unstructured Parsing is the process of extracting usable content from documents that do not follow a fixed schema, such as PDFs, DOCX files, HTML pages, contracts, emails, and research papers.

Q2. Why is Unstructured Parsing important in RAG systems?

It converts raw documents into clean textual content, reducing noise and improving chunking, embeddings, retrieval quality, and final LLM responses.

Q3. What are examples of unstructured data?

PDFs, Word Documents, Emails, HTML Pages, Chat Messages, Contracts, Research Papers.

Q4. What challenges exist in unstructured parsing?

Tables, images, headers, footers, multi-column layouts, and scanned documents.

Q5. What is element-based parsing?

Element-based parsing identifies document components such as titles, paragraphs, tables, and lists instead of treating the whole document as one block of text.

Q6. How does unstructured parsing improve retrieval?

It removes noise, preserves semantic meaning, and produces cleaner chunks, leading to higher retrieval accuracy.

Interview Takeaway

> Unstructured Parsing is the process of extracting meaningful content from documents that do not follow a predefined schema. It is a foundational step in modern RAG systems because enterprise knowledge primarily exists in unstructured formats. High-quality parsing significantly improves chunking, embedding generation, retrieval accuracy, and overall LLM response quality.

2. Structured Parsing

What It Means

Not all data is messy. Some data already comes in organized formats: CSV files with clear columns, JSON with defined fields, SQL tables with schemas, Excel spreadsheets with headers. Structured parsing means reading these well-organized sources and converting them into documents that your RAG pipeline can process.

The key difference from unstructured parsing is that you already know what each piece of data represents. In a CSV file, column headers tell you exactly what each value means. There's no guessing involved.

Why It's Easier (But Still Important)

A CSV row like this:

```
CSV
EmployeeID,Name,Department
101,John,Engineering
102,Alice,Finance
```

...already tells you everything. EmployeeID is 101, the Name is "John", and Department is "Engineering". You don't need layout analysis, OCR, or header detection.

How It Works Internally

The parser follows these steps:

Step 1: Read schema (column headers or field names).

```
TEXT
EmployeeID, Name, Department
```

Step 2: Read each row/record.

```
TEXT
101, John, Engineering
```

Step 3: Map fields to values.

```
TEXT
EmployeeID -> 101
Name -> John
Department -> Engineering
```

Step 4: Create a LangChain Document.

```
PYTHON
Document(
    page_content="Name: John\nDepartment: Engineering",
    metadata={"source": "employees.csv", "row": 0}
)
```

Common Structured Data Sources

- **CSV files:** Customer lists, ticket exports, transaction logs
- **Excel files:** Employee data, financial reports
- **JSON files:** API responses, configuration data
- **Databases:** MySQL, PostgreSQL, SQL Server queries
- **APIs:** REST endpoints returning structured records
- **Data warehouses:** Snowflake, BigQuery, Redshift

Code Examples

CSV Parsing

```
PYTHON
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("tickets.csv")
docs = loader.load()
```

JSON Parsing

Input:

```
JSON
{
  "ticket_id": "T101",
  "priority": "High",
  "description": "Payment Failed"
}
```

The content becomes "Payment Failed" and the rest becomes metadata:

```
PYTHON
metadata = {"ticket_id": "T101", "priority": "High"}
```

Database Parsing

```
SQL
SELECT id, name, department FROM employees
```

Each row becomes a document with fields as content and identifiers as metadata.

Challenges

Structured data is easier, but problems still occur:

- **Missing values:** Empty fields can create incomplete documents.
- **Inconsistent data:** Same department written as "HR", "Human Resources", "HR Team".
- **Duplicate records:** Same data appearing multiple times.
- **Large datasets:** 10 million records may need batch processing.
- **Schema changes:** Column names changing between versions.

Best Practices

- Validate that required fields exist before creating documents.
- Remove duplicates before ingestion.
- Standardize inconsistent values.
- Preserve identifiers (TicketID, EmployeeID) as metadata for traceability.
- Handle null values gracefully - don't create empty documents.

Interview Questions

Q1. What is Structured Parsing?

Structured Parsing is the process of extracting information from data sources that already follow a predefined schema, such as CSV, JSON, Excel, databases, and API responses.

Q2. What are examples of structured data?

CSV Files, JSON Files, Excel Files, Databases, APIs, Data Warehouses.

Q3. Difference between Structured and Unstructured Parsing?

Structured Parsing works with predefined schemas and predictable fields, while Unstructured Parsing extracts information from documents such as PDFs, DOCX files, and HTML pages where structure is not explicitly defined.

Q4. Why is Structured Parsing easier?

The schema already defines relationships, columns, and fields, reducing the need for layout analysis and content interpretation.

Q5. What are common challenges in Structured Parsing?

Missing Data, Duplicate Records, Schema Changes, Inconsistent Values, Large Datasets.

Q6. How is Structured Parsing used in enterprise RAG systems?

It is commonly used to ingest data from databases, CRM systems, ERP systems, APIs, and data warehouses before converting records into Documents.

Interview Takeaway

> Structured Parsing extracts information from data sources with predefined schemas such as CSV files, JSON files, databases, Excel sheets, APIs, CRM systems, and ERP platforms. Because the structure is already known, extraction is highly accurate, predictable, and efficient. In enterprise RAG systems, it's commonly used for ingesting business records, customer information, transactions, and operational data.

3. Metadata Extraction

What It Means

Metadata is "data about data." When you parse a document, you extract not just the content but also descriptive information that helps you find, filter, trace, and govern that content later.

Here's the distinction:

- **Content** answers the user's question: "Passwords must be changed every 90 days."
- **Metadata** helps you find the right content: "This came from security_policy.pdf, page 12, owned by the Security department, version 3.0, classified as Internal."

Why It's Critical

Imagine your vector database contains two conflicting answers:

- Document A says: "Change passwords every 90 days."
- Document B says: "Change passwords every 60 days."

Without metadata, you have no way to know which is current, which department wrote it, or which region it applies to. With metadata, you can filter by version, date, or department to return only the relevant answer.

Metadata also enables:

- **Access control:** Only show documents the user is authorized to see.
- **Source attribution:** Tell users exactly where an answer came from.
- **Filtering:** Search only within a specific department or document type.
- **Lifecycle management:** Identify and retire outdated documents.

Types of Metadata

1. Source Metadata - Where the document came from:

```
PYTHON
{"source": "employee_handbook.pdf", "url": "...", "filepath": "...}"
```

2. Document Metadata - Characteristics of the document:

```
PYTHON
{"page": 10, "language": "English", "document_type": "Policy"}
```

3. Business Metadata - Business context:

```
PYTHON
{"department": "HR", "region": "India", "owner": "HR Team"}
```

4. Technical Metadata - Generated during ingestion:

```
PYTHON
{"chunk_id": "123", "ingestion_date": "2026-08-15"}
```


****5. Security Metadata** - For governance:**

```
PYTHON
{"classification": "Confidential", "access_level": "Internal"}
```

Common Metadata Fields

```
PYTHON
{
  "source": "security_policy.pdf",
  "document_id": "DOC-001",
  "page": 12,
  "department": "Security",
  "category": "Policy",
  "version": "3.0",
  "author": "Security Team",
  "created_date": "2025-01-15",
  "last_modified": "2026-06-01",
  "classification": "Internal",
  "access_level": "All Employees"
}
```

Metadata Extraction Examples**From PDFs (Automatic)**

```
PYTHON
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("security_policy.pdf")
docs = loader.load()
print(docs[0].metadata)
# {"source": "security_policy.pdf", "page": 0}
```

From CSV (Automatic)

```
PYTHON
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("tickets.csv")
docs = loader.load()
print(docs[0].metadata)
# {"source": "tickets.csv", "row": 0}
```

Derived Metadata (From Filename)

Filename: `HR\_LeavePolicy\_India\_2026.pdf`

You can extract:

```
PYTHON
{"department": "HR", "region": "India", "year": "2026"}
```

Explicit Metadata (Manual)

```
PYTHON
from langchain_core.documents import Document

doc = Document(
    page_content=item["content"],
    metadata={
        "document_id": item["id"],
        "department": item["department"],
        "version": item["version"]
    }
)
```

Metadata and Filtering

One of the most powerful uses. When a user asks "Show only HR policies", you filter:

```
PYTHON
filter = {"department": "HR"}
```

Only HR documents are searched, dramatically improving precision.

Metadata and Access Control

```
PYTHON
{"classification": "Confidential"}
```

Only authorized users can access the content. This is essential in banking, healthcare, government, and enterprise systems.

Interview Questions

****Q1. What is Metadata Extraction?****

Metadata Extraction is the process of collecting descriptive information about a document, such as source, author, department, version, category, and classification, during document ingestion.

****Q2. Why is Metadata important in RAG systems?****

Metadata improves search quality, filtering, governance, traceability, and access control while helping retrieve the most relevant document.

****Q3. What is the difference between content and metadata?****

Content contains the actual information, while metadata describes that information and provides contextual details.

****Q4. What are common metadata fields in enterprise systems?****

Source, Document ID, Department, Category, Author, Version, Created Date, Classification, Access Level.

****Q5. What is derived metadata?****

Derived metadata is metadata inferred from filenames, paths, content, or other sources rather than explicitly stored.

****Q6. How does metadata improve retrieval?****

It enables filtering and narrowing search results, reducing irrelevant documents and improving retrieval precision.

****Q7. Can metadata be used for security?****

Yes. Fields like classification and access level help enforce authorization and document governance.

Interview Takeaway

> Metadata Extraction is a critical component of enterprise RAG systems because it enables filtering, governance, traceability, compliance, access control, and retrieval optimization. High-quality metadata improves search accuracy, document management, and overall system reliability, making it just as important as the document content itself.

4. Content Cleaning

What It Means

After parsing, you have raw text. But raw text still contains noise that adds no value to search or retrieval. Content cleaning removes this noise so that only meaningful business content enters your embeddings.

Think of it like preparing ingredients before cooking. Parsing extracts the ingredients from their packaging. Cleaning washes them and removes the inedible parts.

What Gets Removed

Common noise includes:

- **Headers and footers** that repeat on every page ("Company Confidential", "Draft")
- **Page numbers** ("Page 1 of 15")
- **Navigation menus** from web pages ("Home | Products | Contact | About")
- **Cookie notices and ads** from websites
- **Excessive whitespace** and empty lines
- **Duplicate paragraphs** that appear multiple times
- **Boilerplate text** ("All Rights Reserved", "Terms and Conditions Apply")
- **Special characters** that serve no semantic purpose

Types of Content Cleaning

Header/Footer Removal

Before:

```
TEXT
Security Policy
Page 1
Company Confidential
Passwords must be changed every 90 days.
Company Confidential
Page 2
```

After:

```
TEXT
Passwords must be changed every 90 days.
```

Page Number Removal

Before:

```
TEXT
Page 1
Security Guidelines
Page 2
Password Rules
```

After:

```
TEXT
Security Guidelines
Password Rules
```

Whitespace Cleaning

Before:

```
TEXT
Password Policy

MFA Required
```

After:

```
TEXT
Password Policy
MFA Required
```

Navigation Removal (HTML)

Before:

```
TEXT
Home
Products
About Us
Contact
Security Policy
```

After:

```
TEXT
Security Policy
```

Duplicate Content Removal

Before:

```
TEXT
Password Policy
Password Policy
Password Policy
```

After:

```
TEXT
Password Policy
```

Why It Matters for Embeddings

Embedding models convert text into vectors that represent meaning. If your text contains "Page 1", "Confidential", "Company Logo" repeated hundreds of times, the model wastes capacity learning these meaningless patterns.

****Before cleaning:**** The embedding partially represents "Company Confidential" and "Page 5" - useless.

****After cleaning:**** The embedding fully represents actual policy content.

Content Cleaning Pipeline

```

TEXT
Raw Document
|
v
Remove Headers
|
v
Remove Footers
|
v
Remove Page Numbers
|
v
Remove Duplicates
|
v
Remove Empty Lines
|
v
Remove Boilerplate
|
v
Clean Content Ready for Chunking

```

Interview Questions

****Q1. What is Content Cleaning?****

Content Cleaning is the process of removing unnecessary, noisy, invalid, redundant, or irrelevant information from a document before chunking and embedding.

****Q2. Why is Content Cleaning important?****

It ensures embedding models focus on meaningful business content, improving retrieval accuracy and LLM response quality.

****Q3. What are common types of noise?****

Headers, footers, page numbers, navigation menus, advertisements, boilerplate text, excessive whitespace, and duplicate content.

****Q4. How does cleaning improve embeddings?****

By removing noise, the embedding vector represents actual business meaning instead of wasting capacity on repeated, meaningless patterns.

****Q5. What is the difference between Content Cleaning and Content Normalization?****

Content Cleaning removes noise and irrelevant data, while Content Normalization standardizes the remaining content into a consistent representation.

Interview Takeaway

> Content Cleaning removes noisy, repeated, and irrelevant text before chunking and embedding. This ensures only high-quality content enters the retrieval pipeline, directly improving embedding quality, retrieval accuracy, and final LLM responses.

5. Content Normalization

What It Means

Cleaning removes garbage. Normalization makes the remaining content consistent.

Different documents from different teams, systems, and time periods often represent the same information differently. Normalization transforms these variations into a single standard form so that search and retrieval work reliably.

The Problem It Solves

Suppose your knowledge base has documents from three different teams, and they all refer to the same department differently:

```
TEXT
Document A: "HR"
Document B: "Human Resources"
Document C: "Human Resource Dept"
```

When a user searches for HR policies, the retriever might miss documents that say "Human Resource Dept" because the embedding is slightly different. After normalization, all three become "Human Resources" - and retrieval works consistently.

Common Normalization Operations

What	Before	After
Dates	"01-01-2026", "Jan 1 2026"	"2026-01-01"
Countries	"USA", "US", "United States"	"United States"
Departments	"HR", "Human Resources"	"Human Resources"
Abbreviations	"MFA", "Multi Factor Auth"	"Multi-Factor Authentication"
Currency	"Rs. 1000", "INR 1000"	"INR 1000"
Case	"PASSWORD POLICY"	"Password Policy"
Email	"John.Doe@Company.Com"	"john.doe@company.com"
Phone	"9876543210", "+91-9876543210"	"919876543210"
Entities	"MSFT", "Microsoft Corp"	"Microsoft"
Boolean	"Yes", "Y", "True", "1"	"True"

Entity Normalization Example

Before:

```
TEXT
Microsoft
MSFT
Microsoft Corporation
```

After:

```
TEXT
Microsoft
```

The Danger of Over-Normalization

Be careful not to normalize away important differences.

- "Java" and "JavaScript" should NOT both become "Programming Language".
- "Associate", "Employee", and "Contractor" may have different legal meanings.
- "AI" could mean "Artificial Intelligence" or something else in a different domain.

Only normalize when variations genuinely mean the same thing.

Content Normalization Pipeline



Interview Questions

****Q1. What is Content Normalization?****

Content Normalization is the process of transforming cleaned content into a standardized and consistent format before chunking, embedding, and indexing.

****Q2. Why is Content Normalization important in RAG systems?****

It ensures consistent representation of information, improving embedding quality, retrieval accuracy, filtering, and overall search performance.

****Q3. What is the difference between Content Cleaning and Content Normalization?****

Content Cleaning removes noise and irrelevant data, while Content Normalization standardizes the remaining content into a consistent representation.

****Q4. What are common normalization operations?****

Case Normalization, Date Normalization, Number Normalization, Currency Normalization, Entity Normalization, Abbreviation Expansion, Whitespace Normalization.

****Q5. How does normalization improve retrieval?****

It ensures similar concepts are represented consistently, making it easier for retrievers and embedding models to identify related content.

****Q6. What is over-normalization?****

Over-normalization occurs when important distinctions or business meaning are lost because content is standardized too aggressively.

****Q7. What are examples of entity normalization?****

"Microsoft", "MSFT", "Microsoft Corporation" all becoming "Microsoft".

Interview Takeaway

> Content Normalization transforms cleaned content into a standardized, consistent format before chunking and indexing. Common operations include standardizing dates, numbers, currencies, abbreviations, entities, and business terminology. It improves consistency, retrieval accuracy, and embedding quality while reducing semantic fragmentation across large document collections.

6. CharacterTextSplitter

What It Does

This is the simplest chunking method in LangChain. It counts characters and splits the text whenever it reaches the configured limit. It doesn't understand sentences, paragraphs, or meaning - it just counts.

Code Example

```
PYTHON
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=1000,      # max characters per chunk
    chunk_overlap=100,    # characters repeated between chunks
    separator="\n"        # prefer splitting at line breaks
)

chunks = splitter.split_text(text)
```

Core Parameters

- **chunk_size**: Maximum number of characters per chunk (e.g., 1000).
- **chunk_overlap**: How many characters overlap between consecutive chunks (e.g., 100).
- **separator**: Where to prefer splitting (e.g., `"\n"` for line breaks).

How It Works Internally

Document with 5000 characters, `chunk_size=1000`, `chunk_overlap=200`:

```
TEXT
Chunk 1: Characters 1-1000
Chunk 2: Characters 801-1800
Chunk 3: Characters 1601-2600
Chunk 4: Characters 2401-3400
...
```

Example Without Overlap

Text: ``ABCDEFGHIIJ``, `chunk_size=4`, `chunk_overlap=0`

```
TEXT
Chunk 1: ABCD
Chunk 2: EFGH
Chunk 3: IJ
```

No shared context between chunks.

Example With Overlap

Text: `ABCDEFGHIIJ`, chunk_size=4, chunk_overlap=2

```
TEXT
Chunk 1: ABCD
Chunk 2: CDEF
Chunk 3: EFGH
Chunk 4: GHIJ
```

Context carried forward between chunks.

Working with Documents

```
PYTHON
from langchain_core.documents import Document
from langchain_text_splitters import CharacterTextSplitter

docs = [Document(page_content="Password policy details...")]

splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=20)
chunks = splitter.split_documents(docs)
```

The Problem With It

Because it only counts characters, it can cut sentences in half:

```
TEXT
Chunk 1: "Passwords must be changed every 90 da"
Chunk 2: "ys. Multi-factor authentication is required."
```

The first chunk is meaningless on its own.

Advantages

- Simple and easy to understand.
- Very fast processing.
- Good for simple text like logs, notes, and small datasets.
- Configurable with chunk_size, overlap, and separator.

Limitations

- Can split sentences in the middle.
- No semantic awareness whatsoever.
- Not ideal for complex RAG systems.
- Poor context preservation.

Interview Questions

****Q1. What is CharacterTextSplitter?****

CharacterTextSplitter is a LangChain text splitter that divides content into chunks based on a fixed number of characters.

****Q2. What parameters are commonly used?****

```
PYTHON
chunk_size, chunk_overlap, separator
```

****Q3. Does CharacterTextSplitter understand document meaning?****

No. It only counts characters and does not understand sentences, paragraphs, or semantic relationships.

****Q4. What is the biggest limitation of CharacterTextSplitter?****

It can split text in the middle of sentences and paragraphs, leading to loss of context.

****Q5. Why is overlap important?****

Overlap preserves context between adjacent chunks and reduces information loss at chunk boundaries.

****Q6. When should CharacterTextSplitter be used?****

For simple text documents, logs, and smaller datasets where semantic splitting is not critical.

****Q7. What splitter is generally preferred for production RAG systems?****

RecursiveCharacterTextSplitter, because it preserves document structure more effectively.

Interview Takeaway

> CharacterTextSplitter is the simplest text splitter in LangChain. It divides documents based purely on character count. While fast and easy to use, it lacks awareness of sentences, paragraphs, and semantic structure, which can lead to poor chunk quality. It is generally replaced by RecursiveCharacterTextSplitter in production-grade RAG systems.

7. RecursiveCharacterTextSplitter

What It Does

This is the ****default recommended splitter**** for most RAG applications. Instead of blindly counting characters, it tries to split at natural boundaries - first paragraphs, then sentences, then words - before falling back to character-level splitting as a last resort.

Why It's Called "Recursive"

It tries multiple separators in order. If the first separator produces chunks that are too large, it recursively tries the next one.

Default separator hierarchy:

```
PYTHON
["\n\n", "\n", " ", ""]
```

Translation:

1. ****First:**** Try to split at paragraph breaks (`\n\n`). 2. ****If chunk is still too large:**** Split at line breaks (`\n`). 3. ****If still too large:**** Split at spaces (` `). 4. ****Last resort:**** Split at individual characters (`""`).

Code Example

```
PYTHON
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
    separators=["\n\n", "\n", ".", " ", ""]
)

chunks = splitter.split_documents(documents)
```

Full Pipeline Example with PDF

```

PYTHON
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load PDF
loader = PyPDFLoader("security_policy.pdf")
docs = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)

chunks = splitter.split_documents(docs)

```

Pipeline:

```

TEXT
PDF -> PyPDFLoader -> Documents -> RecursiveCharacterTextSplitter -> Chunks

```

Why It's Better

Compare outputs for the same text:

****CharacterTextSplitter**** might produce:

```

TEXT
"MFA is required for all employ"

```

****RecursiveCharacterTextSplitter**** produces:

```

TEXT
"MFA is required for all employees."

```

It respects sentence and paragraph boundaries, so each chunk is more likely to be a complete thought.

Metadata Preservation

When you split documents (not raw text), the original metadata travels with each chunk:

```

PYTHON
# Original document metadata
{"source": "policy.pdf", "page": 10}

# After splitting, each chunk keeps this metadata
{"source": "policy.pdf", "page": 10}

```

This is essential for citations, source attribution, and traceability.

Example Without Overlap

```

PYTHON
chunk_size=100, chunk_overlap=0

```

```

TEXT
Chunk 1: Password Policy
Chunk 2: MFA Policy

```

Potential context loss between chunks.

Example With Overlap

```
PYTHON
chunk_size=100, chunk_overlap=20
```

```
TEXT
Chunk 1: Password Policy + MFA Policy
Chunk 2: MFA Policy + VPN Policy
```

Some context repeated, improving retrieval.

Interview Questions

Q1. What is RecursiveCharacterTextSplitter?

It is the most popular LangChain text splitter that recursively tries paragraph, line, word, and character boundaries to create chunks.

Q2. Why is it called "Recursive"?

Because it tries multiple separators in sequence - if one produces chunks that are too large, it falls back to the next separator recursively.

Q3. What are the default separators?

```
PYTHON
["\n\n", "\n", " ", ""]
```

Q4. Why is it preferred over CharacterTextSplitter?

It preserves natural text boundaries (paragraphs, sentences), producing higher-quality chunks with better retrieval.

Q5. Does it preserve metadata?

Yes. When splitting Document objects, original metadata is preserved on each chunk.

Q6. What are good starting parameters?

chunk_size=1000, chunk_overlap=100-200 is a common starting point for general documents.

Q7. Can separators be customized?

Yes. You can add period (`. `) or other separators to the list for finer control.

Interview Takeaway

> RecursiveCharacterTextSplitter is the most widely used LangChain splitter. It recursively tries paragraph, line, sentence, and word boundaries before falling back to character splitting. This preserves natural text structure and produces higher-quality chunks, making it the standard choice for production RAG systems.

8. TokenTextSplitter

What It Does

LLMs don't process characters - they process **tokens**. A token is roughly a word or word piece. TokenTextSplitter splits text based on actual token count, which aligns directly with how models consume text.

Why Tokens Matter

The same 1000 characters can produce wildly different token counts:

Text A: "Security Policy" - 15 characters, ~2 tokens.

Text B: "Multi-factor-authentication-configuration-settings" - 48 characters, ~8-12 tokens.

Character count is misleading. Token splitting gives predictable chunk sizes relative to model limits.

Code Example

```
PYTHON
from langchain_text_splitters import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=500,      # max tokens per chunk
    chunk_overlap=50     # tokens repeated between chunks
)

chunks = splitter.split_text(text)
```

Requires `tiktoken` for token counting:

```
BASH
pip install tiktoken
```

Working with Documents

```
PYTHON
splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)
```

Metadata is preserved on each chunk.

Internal Working

Document with 2000 tokens, chunk_size=500, chunk_overlap=50:

```
TEXT
Chunk 1: Tokens 1-500
Chunk 2: Tokens 451-950
Chunk 3: Tokens 901-1400
Chunk 4: Tokens 1351-1850
Chunk 5: Tokens 1801-2000
```

Why Token Splitting Is Better for LLM Limits

If a model supports 8192 tokens and you use chunk_size=1000 tokens, you know exactly how many chunks fit in the context window. Character splitting cannot guarantee this.

Advantages

- Token-aware - matches how LLMs actually process text.
- Better context window management.
- Predictable chunk sizes regardless of text complexity.
- Consistent embedding vector distribution.
- Production-friendly for enterprise systems.

Limitations

- Still doesn't understand semantic meaning.
- May still split topics if used alone.
- Often combined with other splitting strategies.

Interview Questions

Q1. What is TokenTextSplitter?

TokenTextSplitter is a LangChain text splitter that divides text based on tokens instead of characters.

Q2. Why is token-based splitting important?

LLMs and embedding models operate on tokens, so token-based chunks have predictable sizes relative to model context windows.

Q3. What library is commonly used for token counting?

```
PYTHON
tiktoken
```

Q4. What are the main parameters?

chunk_size (max tokens per chunk) and chunk_overlap (tokens repeated between chunks).

Q5. When should TokenTextSplitter be used?

When precise control over model context window usage is needed, especially with OpenAI models or strict token limits.

Interview Takeaway

> TokenTextSplitter divides content by token count rather than character count. Since LLMs and embedding models operate on tokens, this gives more predictable and model-aligned chunk sizes. It's particularly useful when working with strict context window limits.

9. MarkdownHeaderTextSplitter

What It Does

When your source content is Markdown (README files, wikis, documentation), this splitter uses the heading structure to create intelligent chunks. Each section becomes its own chunk, and the heading hierarchy becomes metadata.

Code Example

```
PYTHON
from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3")
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

chunks = splitter.split_text(markdown_text)
```

Input Example

```
MARKDOWN
```

```
# Authentication
```

```
Authentication is required before using APIs.
```

```
## JWT Authentication
```

```
JWT tokens are supported.
```

```
## OAuth Authentication
```

```
OAuth 2.0 is also supported.
```

```
# Deployment
```

```
Deploy using Docker.
```

Output Structure

```
PYTHON
```

```
Document(
  page_content="Authentication is required before using APIs.",
  metadata={"Header 1": "Authentication"}
)

Document(
  page_content="JWT tokens are supported.",
  metadata={"Header 1": "Authentication", "Header 2": "JWT Authentication"}
)

Document(
  page_content="OAuth 2.0 is also supported.",
  metadata={"Header 1": "Authentication", "Header 2": "OAuth Authentication"}
)

Document(
  page_content="Deploy using Docker.",
  metadata={"Header 1": "Deployment"}
)
```

Notice how header values become metadata automatically - extremely useful for filtering and navigation.

Handling Multiple Header Levels

```
PYTHON
```

```
headers_to_split_on = [
    ("#", "H1"),
    ("##", "H2"),
    ("###", "H3")
]
```

Output metadata:

```
PYTHON
```

```
{"H1": "Product", "H2": "Authentication", "H3": "JWT"}
```

Very useful for hierarchical retrieval in large documentation.

Best Practice: Combine With RecursiveCharacterTextSplitter

Some Markdown sections can be very long. The recommended production approach:

```
TEXT
Step 1: MarkdownHeaderTextSplitter (preserves structure)
Step 2: RecursiveCharacterTextSplitter (controls chunk size)
```

Common Use Cases

- API Documentation
- Internal Wikis (Confluence)
- README files (GitHub)
- Product Documentation
- Technical Knowledge Bases
- Developer Guides

Interview Questions

Q1. What is MarkdownHeaderTextSplitter?

It is a LangChain text splitter that divides Markdown documents based on header hierarchy rather than character counts.

Q2. Why use Markdown Splitter instead of CharacterTextSplitter?

Because it preserves document structure, headings, and section boundaries, producing better chunks for retrieval.

Q3. What parameter controls splitting?

```
PYTHON
headers_to_split_on
```

Q4. What metadata does Markdown Splitter generate?

Header values become metadata. Example: `{"Header 1": "Authentication", "Header 2": "JWT"}`

Q5. Can Markdown Splitter be combined with RecursiveCharacterTextSplitter?

Yes. This is a common production pattern where documents are first split by headers and then recursively split into smaller chunks.

Q6. When should Markdown Splitter be used?

For documentation, wikis, README files, API guides, and technical knowledge bases written in Markdown.

Q7. What is the biggest advantage of Markdown Splitter?

It preserves document hierarchy and semantic structure while creating meaningful metadata.

Interview Takeaway

> MarkdownHeaderTextSplitter is a structure-aware LangChain text splitter designed for Markdown documents. It uses headers to preserve document hierarchy, generate useful metadata, and create semantically meaningful chunks. In production, it's often combined with RecursiveCharacterTextSplitter to maintain structure while controlling chunk size.

10. HTMLHeaderTextSplitter

What It Does

The HTML equivalent of MarkdownHeaderTextSplitter. It splits HTML documents at heading tags (`<h1>`, `<h2>`, `<h3>`) and uses those headings as metadata.

Code Example

```
PYTHON
from langchain_text_splitters import HTMLHeaderTextSplitter

headers_to_split_on = [
    ("h1", "Header 1"),
    ("h2", "Header 2"),
    ("h3", "Header 3")
]

splitter = HTMLHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on
)

documents = splitter.split_text(html_content)
```

Input Example

```
HTML
<h1>Authentication</h1>
<p>Authentication is required.</p>

<h2>JWT</h2>
<p>JWT Tokens are supported.</p>

<h2>OAuth</h2>
<p>OAuth 2.0 is supported.</p>

<h1>Deployment</h1>
<p>Deploy using Docker.</p>
```

Output Structure

```
PYTHON
Document(
  page_content="Authentication is required.",
  metadata={"Header 1": "Authentication"}
)

Document(
  page_content="JWT Tokens are supported.",
  metadata={"Header 1": "Authentication", "Header 2": "JWT"}
)

Document(
  page_content="OAuth 2.0 is supported.",
  metadata={"Header 1": "Authentication", "Header 2": "OAuth"}
)

Document(
  page_content="Deploy using Docker.",
  metadata={"Header 1": "Deployment"}
)
```

Combining With Recursive Splitting

Extremely common in enterprise systems:

TEXT

Step 1: `HTMLHeaderTextSplitter` (split by HTML headers)

Step 2: `RecursiveCharacterTextSplitter` (split large sections further)

This gives you structure preservation + chunk size control.

Common Use Cases

- Documentation websites and developer portals
- Knowledge bases and help centers
- Internal wikis and portals
- E-learning platforms
- Product support systems and FAQs

Interview Questions

****Q1. What is `HTMLHeaderTextSplitter`?****

It is a `LangChain` splitter that creates chunks using HTML header tags such as `h1`, `h2`, and `h3` rather than character counts.

****Q2. Why use `HTML Splitter` instead of `CharacterTextSplitter`?****

Because it preserves website structure, section boundaries, and semantic organization, leading to better retrieval quality.

****Q3. Which HTML tags are commonly used?***

`<h1>`, `<h2>`, `<h3>`, `<h4>`

****Q4. What metadata does `HTML Splitter` generate?***

Header values become metadata. Example: `{"Header 1": "Authentication", "Header 2": "JWT"}`

****Q5. Can `HTML Splitter` be combined with `RecursiveCharacterTextSplitter`?***

Yes. This is one of the most common production patterns for handling large documentation websites.

****Q6. When should `HTMLHeaderTextSplitter` be used?***

For HTML documents, documentation websites, knowledge bases, help centers, internal portals, and API documentation.

****Q7. What is the biggest advantage of `HTML Splitter`?***

It preserves HTML hierarchy and generates valuable metadata that improves retrieval and navigation.

Interview Takeaway

> `HTMLHeaderTextSplitter` is a structure-aware `LangChain` text splitter for HTML documents. It uses header tags to preserve document hierarchy, generate metadata, and create semantically meaningful chunks. It's commonly used for documentation websites, developer portals, and enterprise knowledge bases, often combined with `RecursiveCharacterTextSplitter`.

11. Code Splitter

What It Does

Source code has its own natural structure: functions, classes, methods, imports. A code-aware splitter understands these boundaries and tries to keep logical code units together rather than splitting a function in half.

The Problem It Solves

A generic text splitter might produce:

```
PYTHON
# Bad Chunk 1:
def reset_password(user_id):
    validated = validate_user(us

# Bad Chunk 2:
er_id)
    if validated:
        send_reset_email(user_id)
```

That's completely useless for code search. A code splitter keeps the full function together:

```
PYTHON
# Good Chunk (complete function):
def reset_password(user_id):
    validated = validate_user(user_id)
    if validated:
        send_reset_email(user_id)
```

Code Example

```
PYTHON
from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(source_code)
```

Supported Languages

```
PYTHON
Language.PYTHON
Language.JAVA
Language.JS
Language.TS
Language.CPP
Language.GO
Language.PHP
Language.RUST
```

Python Code Example

Original:

```
PYTHON
class UserService:
    def login(self):
        pass

    def logout(self):
        pass

class PaymentService:
    def pay(self):
        pass
```

Code Splitter creates:

```
TEXT
Chunk 1: class UserService (login, logout)
Chunk 2: class PaymentService (pay)
```

Instead of breaking functions arbitrarily.

Java Example

```
JAVA
public class UserService {
    public void login() { }
    public void logout() { }
}
```

Using `Language.JAVA`, the splitter understands class and method boundaries.

Use Cases

- **Repository Q&A:** "Where is password reset implemented?"
- **Code search:** Find functions by description.
- **Developer assistants:** GitHub Copilot-style systems.
- **Legacy modernization:** Analyze old codebases.
- **Software documentation:** Code-aware knowledge bases.

Best Practice: Store File Metadata

```
PYTHON
metadata = {"file": "auth.py", "repository": "project-a"}
```

Interview Questions

Q1. What is a Code Splitter?

A language-aware text splitter that chunks source code based on programming constructs such as functions, classes, methods, and modules.

Q2. Why can't we use CharacterTextSplitter for code?

Because it may split code in the middle of functions, classes, or statements, reducing retrieval and embedding quality.

Q3. How does LangChain support code chunking?

Using language-aware splitters created through `RecursiveCharacterTextSplitter.from\_language()`.

Q4. Which languages are commonly supported?

Python, Java, JavaScript, TypeScript, C++, Go, PHP, Rust.

Q5. Why is code chunking important for Code RAG systems?

It preserves logical code units, improves embeddings, and enables accurate retrieval of functions, classes, and implementation

details.

****Q6. What is the biggest advantage of Code Splitter?****

It preserves programming language structure rather than blindly splitting text.

****Q7. Where is Code Splitter commonly used?****

GitHub Repository Chatbots, Developer Assistants, Enterprise Code Search, Software Documentation Systems, Code Intelligence Platforms.

Interview Takeaway

> Code Splitter is a language-aware text splitting strategy for source code ingestion. Unlike generic splitters, it preserves logical code structures such as functions, classes, methods, and modules, making it ideal for code search, repository Q&A systems, and developer assistants.

12. Semantic Chunking

What It Does

This is the most advanced chunking technique. Instead of using size limits or document structure, semantic chunking uses **meaning** to decide where to split. It groups related sentences together and creates a boundary when the topic changes.

How It Works

The process:

```

TEXT
Document
  |
  v
Split into sentences
  |
  v
Generate embedding for each sentence
  |
  v
Measure similarity between neighboring sentences
  |
  v
Detect where similarity drops (topic change)
  |
  v
Create chunk boundaries at topic changes

```

Code Example

```

PYTHON
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(OpenAIEmbeddings())
documents = chunker.create_documents([text])

```

When It Shines

Consider this text:

TEXT

Passwords must be changed every 90 days.
 Multi-factor authentication is mandatory.
 VPN access requires manager approval.
 Employees receive 25 days of annual leave.
 Health insurance covers family members.

****Fixed-size splitter**** might group security policies with HR benefits in the same chunk.

****Semantic chunking**** correctly identifies two topics:

- ****Security chunk:**** Password, MFA, VPN rules
- ****HR chunk:**** Leave, insurance benefits

Each chunk now represents one coherent topic, making embeddings much more meaningful.

Semantic Similarity Concept

Each sentence gets an embedding vector. If neighboring sentences have high similarity, they stay together. When similarity drops, a boundary is created.

TEXT

"Passwords must change..." -> Security embedding
 "MFA is required..." -> Security embedding (high similarity -> same chunk)
 "Employees get leave..." -> HR embedding (low similarity -> NEW chunk)

The Trade-Off

Semantic chunking produces better chunks, but it's more expensive because it requires generating embeddings during the chunking process itself.

- ****Character splitting:**** Just count characters. Very cheap.
- ****Semantic chunking:**** Generate embeddings, calculate similarities, identify boundaries. Requires additional computation.

For high-value documents (policies, contracts, research papers), the extra cost is worth it. For millions of low-value documents, simpler methods are more practical.

Common Use Cases

- Enterprise knowledge bases (policies, procedures, SOPs)
- Legal systems (contracts, compliance documents)
- Research systems (papers, patents)
- Healthcare (medical guidelines, clinical documentation)
- Financial systems (regulatory documents, auditing standards)

Interview Questions

****Q1. What is Semantic Chunking?****

A chunking strategy that creates chunks based on meaning and topic boundaries rather than fixed character or token counts.

****Q2. How does Semantic Chunking work?****

It generates embeddings for text segments, measures semantic similarity, detects topic changes, and creates boundaries where meaning shifts.

****Q3. Why is Semantic Chunking better than Character Chunking?****

Because it keeps related concepts together and avoids splitting content in the middle of a topic.

****Q4. Why is Semantic Chunking expensive?****

It requires embedding generation and similarity calculations during the chunking process.

****Q5. When should Semantic Chunking be used?****

For high-value documents such as policies, contracts, technical documentation, research papers, and enterprise knowledge

bases where retrieval quality is critical.

****Q6. What is the biggest advantage of Semantic Chunking?****

It produces semantically coherent chunks that significantly improve retrieval accuracy and embedding quality.

****Q7. What is the biggest limitation of Semantic Chunking?****

Higher computational cost and slower ingestion compared to traditional chunking methods.

Interview Takeaway

> Semantic Chunking groups content based on meaning rather than character counts or formatting. It uses embeddings and semantic similarity to identify topic boundaries. Although more expensive, it produces superior retrieval quality, better embeddings, and more accurate RAG responses, making it increasingly popular for enterprise-grade AI systems.

13. Chunk Size: The Most Important Parameter

What It Controls

Chunk size is the maximum amount of content allowed in a single chunk. It's the single most impactful parameter in your entire RAG pipeline because it directly affects how precise or how contextual your retrieval is.

The Fundamental Trade-Off

****Small chunks (100-300 tokens):****

- Very precise retrieval - you get exactly the sentence that answers the question.
- But you lose surrounding context that might be needed for a complete answer.
- You create more vectors, increasing storage costs.

****Large chunks (1500-3000 tokens):****

- Rich context - each chunk contains full explanations.
- But retrieval becomes less precise - you might pull back 2000 tokens when only 50 are relevant.
- Higher token costs when sending context to the LLM.

Practical Guidelines

Content Type	Recommended Size	Why
FAQs	100-300 tokens	Questions are naturally small
Policies	500-1000 tokens	Need moderate context
Technical docs	800-1500 tokens	Concepts span paragraphs
Research papers	1000-2000 tokens	Complex explanations
Source code	300-800 tokens	Functions should stay whole

Chunk Size with RecursiveCharacterTextSplitter

```
PYTHON
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
```

Chunk Size with TokenTextSplitter

```
PYTHON
splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
```

How to Choose (Chunk Size Tuning)

There is no universal "best" chunk size. The process is iterative:

```
TEXT
1. Choose a starting size (500-1000)
2. Test retrieval with real queries
3. Measure retrieval accuracy
4. If chunks are too narrow (missing context) -> increase size
5. If chunks have too much noise -> decrease size
6. Repeat until quality is satisfactory
```

Common Mistakes

- **Too small (chunk_size=20):** Context completely lost.
- **Too large (chunk_size=10000):** Poor retrieval precision, too much noise.
- **Same size for everything:** Policies and source code need different sizes.
- **Never testing:** Chunk size should be measured using actual retrieval performance.

Impact on Storage and Cost

```
TEXT
100,000 token document with chunk_size=100 -> 1000 chunks
100,000 token document with chunk_size=1000 -> 100 chunks
```

Smaller chunks = more embeddings = more storage = higher cost.

Interview Questions

Q1. What is Chunk Size?

Chunk Size is the maximum amount of content allowed in a chunk during text splitting.

Q2. Why is Chunk Size important?

It directly impacts retrieval quality, embedding quality, context preservation, storage requirements, and token consumption.

Q3. What happens when Chunk Size is too small?

Context may be lost, and too many chunks are created, increasing storage and embedding costs.

Q4. What happens when Chunk Size is too large?

Chunks may contain multiple topics, reducing retrieval precision and increasing irrelevant context.

Q5. Is there a perfect chunk size?

No. The optimal chunk size depends on document type, embedding model, retriever, and business use case.

Q6. What is the most common chunk size for enterprise RAG systems?

A common starting point is 500-1000, followed by tuning based on retrieval performance.

Q7. Why should chunk size be tested?

Because chunk size significantly influences RAG quality and there is no universal value that works for every dataset.

Interview Takeaway

> Chunk Size is one of the most critical parameters in a RAG pipeline. Small chunk sizes improve retrieval precision but may lose context, while large chunk sizes preserve context but reduce precision. Successful enterprise RAG systems carefully tune chunk size based on document type, embedding model, and retriever behavior to achieve the right balance.

14. Chunk Overlap: Preserving Context at Boundaries

What It Controls

Chunk overlap is the amount of content intentionally repeated between consecutive chunks. It exists to prevent information loss at chunk boundaries.

Why Boundaries Are a Problem

When you split a document, important information sometimes falls right at the cut point:

```
TEXT
Original: "Multi-Factor Authentication is required for all employees."

Without overlap:
  Chunk 1: "Multi-Factor Authentication is required"
  Chunk 2: "for all employees. Tokens expire after 12 hours."
```

If a user asks "Who needs MFA?", the answer ("all employees") is separated from the question context ("MFA is required"). With overlap, both chunks contain the full sentence.

How It Works

Document: `1 2 3 4 5 6 7 8 9 10`, chunk_size=5, chunk_overlap=2:

```
TEXT
Chunk 1: 1 2 3 4 5
Chunk 2: 4 5 6 7 8
Chunk 3: 7 8 9 10
```

Repeated content: `4 5` and `7 8` bridge the chunks.

With RecursiveCharacterTextSplitter

```
PYTHON
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150
)
```

With TokenTextSplitter

```
PYTHON
splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
```

Practical Settings

A good rule of thumb is **10-20% of your chunk size**:

Chunk Size	Recommended Overlap
300 (small)	30-50
1000 (medium)	100-200
2000 (large)	200-400

Too Little Overlap

```
PYTHON
chunk_overlap = 0
```

Problems: Context gets lost at boundaries. Related sentences get separated. Retrieval misses connections.

Too Much Overlap

```
PYTHON
chunk_overlap = 800 # with chunk_size = 1000
```

Problems: Massive duplication. More embeddings and storage. Higher cost. Slower retrieval with diminishing quality gains.

Chunk Size vs Chunk Overlap - Working Together

```
PYTHON
chunk_size = 1000    # How much content enters a chunk
chunk_overlap = 100   # How much content is repeated between chunks
```

These two parameters are always tuned together. They represent the trade-off between context completeness and storage efficiency.

Common Mistakes

- **No overlap (0):** Can hurt retrieval at boundaries.
- **Huge overlap** (e.g., 500 for chunk_size=600): Excessive duplication, wasted resources.
- **Same overlap for all documents:** Different content types need different settings.
- **Never testing:** Overlap should be evaluated using real queries.

Typical Production Settings

```
PYTHON
# General documents
chunk_size=1000, chunk_overlap=100

# Technical documentation
chunk_size=1200, chunk_overlap=150

# Policies
chunk_size=800, chunk_overlap=100

# Source code
chunk_size=500, chunk_overlap=50
```

Interview Questions

****Q1. What is Chunk Overlap?****

Chunk Overlap is the amount of content intentionally repeated between consecutive chunks to preserve context across chunk boundaries.

****Q2. Why is overlap important?****

It preserves context near chunk boundaries. Without overlap, important information at the split point may be lost, reducing retrieval quality.

****Q3. What happens with too little overlap?****

Context gets lost at boundaries, and related sentences may be separated across chunks.

****Q4. What happens with too much overlap?****

Excessive duplication, more embeddings, more storage, higher costs, and slower retrieval.

****Q5. What is a good starting overlap value?****

10-20% of chunk size. For chunk_size=1000, use chunk_overlap=100-200.

****Q6. How do chunk size and overlap work together?****

Chunk size controls how much content enters a chunk. Overlap controls how much is repeated. They are always tuned together.

****Q7. Should the same overlap be used for all document types?****

No. Different content types (policies, code, documentation) benefit from different overlap values.

Interview Takeaway

> Chunk Overlap repeats content between adjacent chunks to preserve context at boundaries. It prevents information loss when important content spans the split point. A common guideline is 10-20% of chunk size. Too little loses context; too much creates expensive duplication.

15. Putting It All Together: Production Strategy

The Recommended Pipeline

For a typical enterprise RAG system:

```
PYTHON
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Parse the document
loader = PyPDFLoader("security_policy.pdf")
documents = loader.load()

# 2. Configure intelligent chunking
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
    separators=["\n\n", "\n", ".", " ", ""]
)

# 3. Create chunks (metadata is preserved automatically)
chunks = splitter.split_documents(documents)

# 4. From here: embed chunks and store in vector database
```

Decision Guide: Which Splitter to Use

Situation	Best Splitter
General documents, policies	RecursiveCharacterTextSplitter
Markdown documentation	MarkdownHeaderTextSplitter + Recursive
HTML websites	HTMLHeaderTextSplitter + Recursive
Source code	Language-aware Code Splitter
High-value contracts, research	SemanticChunker
Need precise token control	TokenTextSplitter
Simple logs, flat text	CharacterTextSplitter

Key Takeaways for Interviews

1. **Parsing quality determines everything.** Bad parsing means bad embeddings, bad retrieval, bad answers. 2. **Metadata is not optional.** It enables filtering, access control, citations, and governance. 3. **Cleaning and normalization prevent noise** from polluting your vector space. 4. **RecursiveCharacterTextSplitter is your default.** Use it unless you have a specific reason to choose something else. 5. **Chunk size is a trade-off** between precision and context. Always tune it with real queries. 6. **Overlap prevents boundary information loss.** Use 10-20% of your chunk size. 7. **Structure-aware splitters** (Markdown, HTML, Code) preserve the original document organization. 8. **Semantic chunking** gives the best quality but at higher cost - reserve it for important content. 9. **Token splitting** aligns with model limits - use it when context window management is critical. 10. **Always test with real queries.** No parameter value is universally correct.